

Sistemi Operativi

Ripasso G2 — Domande teoriche

27 compiti analizzati · Argomenti ordinati per frequenza

#	Argomento	Freq.
1	RAID confronti	12×
2	Capability/ACL — revoca e crittografia	11×
3	i-node: nome file + link count errato	10×
4	Banchiere: quando si esegue + safe/unsafe	10×
5	LRU hardware / vs Second Chance	8×
6	Salt nelle password	8×
7	Aging + scheduler interattivi	8×
8	Banchiere: limiti pratici	7×
9	Spinlock in sistemi SMP	7×
10	Stack property → no anomalia Belady	6×
11	Paginazione: perché si usa per MV	6×
12	Concatenata vs indicizzata (accesso diretto)	6×
13	System call: meccanismo e parametri	6×

1. RAID confronti

Frequenza: 12× — quasi certo

D: Quanti dischi coinvolge una scrittura di un blocco in RAID 5? Perché?

4 dischi: i dati vengono distribuiti a stripe su N-1 dischi e la parità è scritta sul disco rimanente (ruotando). Per aggiornare la parità bisogna: (1) leggere il vecchio dato, (2) leggere la vecchia parità, (3) scrivere il nuovo dato, (4) scrivere la nuova parità. Quindi una scrittura coinvolge sempre 4 accessi su 2 dischi distinti.

D: RAID 0 vs RAID 5: quale ha tempo medio di accesso minore? Discutere lettura e scrittura.

RAID 0 è più veloce in scrittura (nessun overhead di parità). In lettura entrambi sono simili (striping parallelizza). RAID 5 ha overhead in scrittura (4 accessi invece di 1), ma tollera 1 disco guasto. RAID 0 non tollera nessun guasto e non è adatto a dati importanti.

D: RAID 1 vs RAID 5: quale array è statisticamente più veloce in lettura?

RAID 1 (mirroring): le letture possono essere distribuite sui due dischi mirror, quindi può leggere da quello con la testina più vicina. RAID 5: lettura parallelizzata su tutti gli N dischi. Con molti dischi RAID 5 può essere più veloce in lettura, ma RAID 1 è più semplice e con 2 dischi offre throughput doppio.

D: RAID 10 vs RAID 5 su 4 dischi da 1 GiB: quale memorizza più dati?

RAID 5 con 4 dischi: capacità = 3 GiB (N-1 dischi). RAID 10 con 4 dischi: capacità = 2 GiB (metà per mirroring). Quindi RAID 5 memorizza più dati. RAID 10 però tollera più guasti in modo più robusto e ha scritture più veloci.

D: Come funziona RAID 60? Numero minimo di dischi, capacità, dischi per una scrittura.

RAID 60 = RAID 6 striping su più gruppi RAID 0 (in realtà: stripe di gruppi RAID 6). Minimo 8 dischi (2 gruppi da 4). Ogni gruppo RAID 6 usa 2 dischi per doppia parità. Una scrittura coinvolge i dischi del gruppo colpito dallo stripe: 6 accessi per aggiornare le due parità (leggi dato vecchio, leggi P, leggi Q, scrivi dato, scrivi P, scrivi Q).

D: RAID 61 vs RAID 16: quanti dischi possono guastarsi?

RAID 61 = RAID 1 di array RAID 6: ogni array RAID 6 tollera 2 guasti, e il mirroring ne tollera un altro intero array. Fault tolerance altissima. RAID 16 = RAID 6 di array RAID 1: ogni coppia mirror tollera 1 guasto, RAID 6 sopra tollera 2 array morti. Anche qui fault tolerance molto elevata ma diversa struttura.

D: LOOK vs C-LOOK: quando si preferisce C-LOOK?

LOOK va avanti e indietro servendo le richieste nelle due direzioni. C-LOOK va in una sola direzione e poi riparte dall'inizio. C-LOOK si preferisce quando si vuole un tempo di attesa massimo più prevedibile e uniforme: con LOOK le richieste agli estremi del disco aspettano più a lungo. C-LOOK garantisce un tempo di attesa proporzionale alla distanza logica dall'estremo.

2. Capability/ACL — revoca e crittografia

Frequenza: 11× — quasi certo

D: Perché revocare una capability è più difficile che revocare un diritto in una ACL?

Con una ACL il diritto è centralizzato: basta modificare la lista associata alla risorsa. Con una capability il token è distribuito nei processi/utenti: non esiste un posto unico dove revocarla. Bisogna trovare e invalidare tutte le copie del token, il che è difficile o impossibile se le capability sono state copiate o passate ad altri processi.

D: Quando è sconsigliato usare un meccanismo di protezione di tipo ACL?

Quando il numero di soggetti (utenti/processi) è molto grande e dinamico: mantenere la lista aggiornata diventa oneroso. Anche quando l'accesso è molto distribuito o quando si vuole delegare facilmente un diritto ad altri (le capability sono più adatte alla delega).

D: Per memorizzare capability lato utente occorre crittografia simmetrica o asimmetrica? Perché?

Occorre crittografia asimmetrica (chiave pubblica). Il kernel firma la capability con la propria chiave privata. Il processo utente possiede la capability firmata ma non può falsificarla perché non conosce la chiave privata. Con crittografia simmetrica il processo utente dovrebbe conoscere la chiave per verificare, e quindi potrebbe anche falsificare la capability.

D: Come è possibile che un buffer overflow permetta l'esecuzione di codice iniettato?

Il buffer è sullo stack. Overflow sovrascrive il return address del frame corrente. L'attaccante inserisce codice macchina nel buffer e sovrascrive il return address con l'indirizzo del buffer stesso. Quando la funzione ritorna, la CPU salta al codice iniettato ed lo esegue. Questo è possibile se lo stack è eseguibile (oggi i sistemi moderni usano NX bit / DEP per impedirlo).

3. i-node: nome file + link count errato

Frequenza: 10× — quasi certo

D: Perché il nome del file non è memorizzato nell'i-node?

Perché un file può avere più nomi (hard link): più directory entry possono puntare allo stesso i-node. Se il nome fosse nell'i-node, un file potrebbe avere un solo nome. Separando nome (nella directory entry) e metadati (nell'i-node) si consente di più nomi di riferirsi allo stesso file.

D: Cosa succede se il link count nell'i-node è errato per eccesso?

Il file non viene mai cancellato anche quando tutte le directory entry che lo referenziano vengono eliminate. Il contatore non arriva mai a zero, quindi il filesystem non libera i blocchi. Si ha una perdita di spazio (blocchi occupati da file irraggiungibili).

D: Cosa succede se il link count è errato per difetto?

Il file viene cancellato prematuramente: quando si elimina una directory entry il contatore scende a zero prima del dovuto, e il filesystem dealloca i blocchi del file. Gli altri hard link che puntano all'i-node diventano dangling reference e accedendo al file si ottiene un errore o dati corrotti.

D: Dove è memorizzato il nome del file in ext2?

Nelle directory entry, che sono strutture memorizzate nei blocchi dati delle directory. Ogni entry contiene: numero di i-node, lunghezza dell'entry, lunghezza del nome, e il nome del file.

4. Banchiere: quando si esegue + safe/unsafe

Frequenza: 10× — quasi certo

D: Quali eventi causano l'esecuzione dell'algoritmo del Banchiere?

L'algoritmo viene eseguito ogni volta che un processo richiede risorse (non alla restituzione). Prima di assegnare le risorse richieste, il SO simula lo stato che si otterrebbe assegnandole e verifica se è safe.

D: Cosa fa il SO se lo stato risulta safe? E se risulta unsafe?

Safe: le risorse vengono assegnate al processo e l'esecuzione continua normalmente. Unsafe: le risorse NON vengono assegnate. La richiesta viene sospesa (il processo rimane bloccato in attesa). Quando un altro processo restituisce risorse, si rivaluta se le richieste sospese possono ora essere soddisfatte mantenendo lo stato safe.

D: Qual è la complessità computazionale del Banchiere monovaluta e multivaluta?

Monovaluta: $O(n^2)$ dove n è il numero di processi (algoritmo di ricerca della sequenza safe).

Multivaluta: $O(n^2 \times r)$ dove r è il numero di tipi di risorse (per ogni passo si confrontano vettori di dimensione r).

Nota: La descrizione errata comune: 'quando un processo restituisce risorse si controlla se il nuovo stato consente di soddisfare richieste sospese'. È errata perché il banchiere va eseguito simulando l'assegnazione, non alla restituzione.

5. LRU hardware / vs Second Chance

Frequenza: 8× — molto probabile

D: Perché implementare LRU richiede hardware molto complesso e costoso?

LRU richiede di conoscere l'ordine esatto degli accessi a tutte le pagine in memoria. Servirebbero: un timestamp aggiornato ad ogni accesso a ogni pagina (non solo ai page fault), oppure una lista ordinata mantenuta in hardware. Con N frame si avrebbe bisogno di una matrice $N \times N$ di bit aggiornata ad ogni riferimento, il che è impraticabile per N grande (es. milioni di frame).

D: Perché nei sistemi reali si preferisce Second Chance (clock) a LRU?

Second Chance approssima LRU usando solo il reference bit (R) che la MMU aggiorna automaticamente ad ogni accesso. Non richiede hardware aggiuntivo. L'algoritmo scorre i frame

circolarmente: se $R=1$ lo azzera e passa avanti, se $R=0$ quella è la vittima. È molto più semplice da implementare pur dando risultati pratici simili a LRU.

Nota: Second Chance non è a stack (può soffrire anomalia di Belady), ma nella pratica funziona bene e il suo costo di implementazione è trascurabile.

6. Salt nelle password

Frequenza: 8× — molto probabile

D: Da quale tipo di attacco protegge il meccanismo del sale (salt)?

Protegge dagli attacchi a dizionario pre-calcolati (rainbow table). Senza sale, un attaccante può pre-calcolare gli hash di tutte le password comuni e confrontarli con il file delle password. Con il sale, ogni password ha un hash diverso anche se la password è uguale, rendendo inutili le rainbow table pre-calcolate.

D: Perché un sale di un solo byte può non essere sufficiente?

Con 1 byte ci sono solo 256 valori possibili di sale. Un attaccante può pre-calcolare le rainbow table per tutti e 256 i valori di sale con costo moltiplicato per 256, che rimane computazionalmente fattibile. Un sale di 16+ byte rende il precalcolo praticamente impossibile.

D: A cosa serve e come funziona il meccanismo challenge/response?

Serve per autenticare senza trasmettere la password in chiaro. Il server invia una sfida (numero casuale). Il client calcola una risposta applicando una funzione crittografica alla sfida usando la password come chiave. Il server verifica la risposta. La password non transita mai sulla rete, e ogni sessione usa una risposta diversa (non riproducibile).

7. Aging + scheduler interattivi

Frequenza: 8× — molto probabile

D: A cosa serve l'aging e come funziona?

L'aging serve a evitare la starvation negli scheduler a priorità. Un processo a bassa priorità potrebbe non essere mai eseguito se arrivano continuamente processi ad alta priorità. Con l'aging, la priorità di un processo aumenta gradualmente con il tempo di attesa. Dopo un certo tempo il processo avrà priorità sufficiente per essere schedulato.

D: Quali sono gli scheduler più adatti a sistemi interattivi: preemptive o non-preemptive?

Preemptive. In un sistema interattivo è fondamentale che il SO possa interrompere un processo in esecuzione per rispondere agli input dell'utente. Con scheduler non-preemptive un processo CPU-bound monopolizza la CPU e i processi interattivi devono aspettare, producendo latenze inaccettabili.

D: Quali effetti collaterali ha dimezzare il time slice in Round Robin?

Pro: i processi interattivi vengono serviti più rapidamente. Contro: aumenta il context switch overhead. Con time slice molto piccolo, la CPU passa più tempo a fare context switch che a eseguire codice utile (overhead di sistema cresce). Nel caso estremo (time slice $\rightarrow 0$) il sistema passa tutto il tempo a switchare.

D: Perché è necessario l'interval timer per implementare lo scheduler Round Robin?

Senza interval timer, un processo potrebbe non cedere mai volontariamente la CPU. L'interval timer genera un interrupt periodico che permette al SO di riprendere il controllo della CPU alla scadenza del time slice, indipendentemente da ciò che sta eseguendo il processo.

8. Banchiere: limiti pratici

Frequenza: 7× — molto probabile

D: Perché l'algoritmo del Banchiere viene raramente usato nei sistemi reali?

- Richiede la dichiarazione anticipata del massimo credito di ogni processo: nella realtà i processi raramente conoscono in anticipo quante risorse useranno al massimo.
 - Il numero di processi e di tipi di risorse deve essere fisso e noto a priori: nei sistemi reali entrambi cambiano dinamicamente.
 - Costo computazionale: $O(n^2 \times r)$ ad ogni richiesta di risorse, troppo costoso per sistemi con molti processi.
 - Conservativo: può bloccare richieste che non avrebbero portato a deadlock, degradando le prestazioni inutilmente.
-

D: L'algoritmo del Banchiere evita anche situazioni di starvation?

No. Il Banchiere evita il deadlock ma non garantisce l'assenza di starvation. Un processo potrebbe rimanere bloccato indefinitamente se le sue richieste portano sempre a stati unsafe, anche se altri processi continuano ad eseguire. Servirebbero meccanismi aggiuntivi (es. aging) per garantire l'assenza di starvation.

9. Spinlock in sistemi SMP

Frequenza: 7× — molto probabile

D: Perché gli spinlock sono necessari nei kernel SMP (simmetrici multiprocessore)?

In un sistema SMP più CPU eseguono codice kernel contemporaneamente. Per proteggere strutture dati condivise del kernel (es. lista dei processi ready, tabelle di memoria) servono meccanismi di mutua esclusione. Gli spinlock usano istruzioni atomiche hardware (test&set, compare&swap;) che funzionano correttamente anche con più CPU sulla stessa memoria. I semafori o i meccanismi basati su disabilitazione degli interrupt non bastano: disabilitare gli interrupt su una CPU non impedisce all'altra CPU di accedere alla stessa struttura dati contemporaneamente.

D: Perché non si possono usare i semafori al posto degli spinlock nel kernel SMP?

I semafori mettono il processo in sleep se la risorsa non è disponibile, il che richiede un context switch. Nel kernel, durante una sezione critica, il context switch potrebbe essere impossibile o pericoloso. Gli spinlock invece fanno busy-waiting: la CPU rimane attiva e ricontrolla continuamente. Per sezioni critiche brevi (come accedere a una lista del kernel) questo è preferibile al costo di un context switch.

10. Stack property → no anomalia Belady

Frequenza: 6× — probabile

D: Cos'è la stack property per un algoritmo di rimpiazzamento?

Un algoritmo ha la stack property se l'insieme delle pagine in memoria con n frame è sempre un sottoinsieme dell'insieme con $n+1$ frame, per qualsiasi stringa di riferimenti. Formalmente: $S(n) \subseteq S(n+1)$ per ogni t .

D: Dimostrare che un algoritmo a stack non può soffrire dell'anomalia di Belady.

Supponiamo per assurdo che con $n+1$ frame si abbiano più page fault che con n frame. Per la stack property, ogni pagina in memoria con n frame è anche in memoria con $n+1$ frame ($S(n) \subseteq S(n+1)$). Quindi ogni riferimento che non causa page fault con n frame non lo causa nemmeno con $n+1$ frame. Ne segue che i page fault con $n+1$ frame non possono essere più di quelli con n frame: contraddizione.

D: LRU ha la stack property? E FIFO?

LRU: sì, ha la stack property. MIN: sì. FIFO: no, non ha la stack property, per questo può soffrire dell'anomalia di Belady (come dimostra la stringa classica 1,2,3,4,1,2,5,1,2,3,4,5 con 3 e 4 frame).

11. Paginazione: perché si usa per la memoria virtuale

Frequenza: 6× — probabile

D: Perché viene usata la paginazione per implementare la memoria virtuale?

- Elimina la frammentazione esterna: i frame fisici possono essere assegnati a qualsiasi pagina logica, non serve memoria contigua.
- Permette di caricare solo le pagine necessarie (demand paging): un processo può usare più memoria di quella fisicamente disponibile.
- Semplifica la gestione della memoria: tutte le pagine hanno la stessa dimensione, quindi l'allocazione è semplice e veloce.
- Permette la condivisione: più processi possono condividere pagine fisiche (es. codice di librerie).
- Isola i processi: ogni processo ha il proprio spazio di indirizzi logico, non può accedere alla memoria di altri processi.

D: Perché la paginazione è preferita alla compattazione?

La compattazione richiede di spostare fisicamente i processi in memoria (costoso, $O(n)$ movimenti di dati) e di sospendere l'esecuzione. La paginazione gestisce la frammentazione esterna senza spostare dati: basta aggiornare le tabelle delle pagine.

12. Allocazione concatenata vs indicizzata (accesso diretto)

Frequenza: 6× — probabile

D: Per accedere direttamente (seek) a un file di grandi dimensioni è più efficiente l'allocazione concatenata o indicizzata?

Indicizzata (come ext2 con i-node). Con allocazione concatenata, per raggiungere il blocco N bisogna seguire la catena di puntatori da inizio file: $O(N)$ accessi a disco. Con allocazione indicizzata, l'i-node contiene puntatori diretti, indiretti, doppiamente indiretti: l'accesso è $O(1)$ per file piccoli e $O(\log N)$ per file grandi (al massimo 3 livelli di indirezione).

D: Perché ext2 è più efficiente per file piccoli che per file grandi?

L'i-node contiene 12 puntatori diretti: per file piccoli (fino a 12 blocchi) l'accesso è diretto, un solo accesso alla tabella. Per file grandi servono blocchi di indirezione singola, doppia o tripla: ogni livello di indirezione richiede un accesso a disco aggiuntivo per leggere il blocco di puntatori.

D: Come si calcola la lunghezza massima di un file in ext2?

Con blocchi da 1KB e puntatori a 4 byte: 256 puntatori per blocco. • 12 diretti: 12 KB • 1 indiretto singolo: 256 KB • 1 indiretto doppio: 256^2 KB = 64 MB • 1 indiretto triplo: 256^3 KB = 16 GB Totale ≈ 16 GB (con blocchi da 4KB il massimo sale a ~ 4 TB).

13. System call: meccanismo e parametri

Frequenza: 6× — probabile

D: Qual è la differenza fra una funzione di libreria e una system call?

Una funzione di libreria (es. printf, malloc) è codice utente che gira in modalità utente nello spazio di indirizzamento del processo. Una system call è un'interfaccia verso il kernel: la CPU passa in modalità kernel (privilegiata) per eseguire operazioni che il codice utente non può fare direttamente (accesso a hardware, gestione processi, I/O).

D: Come sono realizzate le funzioni wrapper della libc (es. open, read)?

Sono funzioni C che preparano i parametri della system call nei registri del processore (o sullo stack, secondo la calling convention del SO), poi eseguono un'istruzione trap (es. syscall su x86-64, int 0x80 su x86). La CPU passa in modalità kernel, il kernel identifica la system call dal numero nel registro apposito, esegue il codice kernel, poi ritorna in modalità utente mettendo il valore di ritorno nel registro apposito (es. rax su x86-64).

D: Come vengono passati i parametri e il valore di ritorno delle system call?

I parametri vengono passati nei registri del processore (su Linux x86-64: rdi, rsi, rdx, r10, r8, r9 per i primi 6 parametri). Il numero della system call va in rax. Il valore di ritorno viene messo in

rax dal kernel prima di tornare in modalità utente. In caso di errore, rax contiene un valore negativo (il codice di errore negato) e la libc lo converte in errno.